



SI100B: Introduction to Information Science and Technology

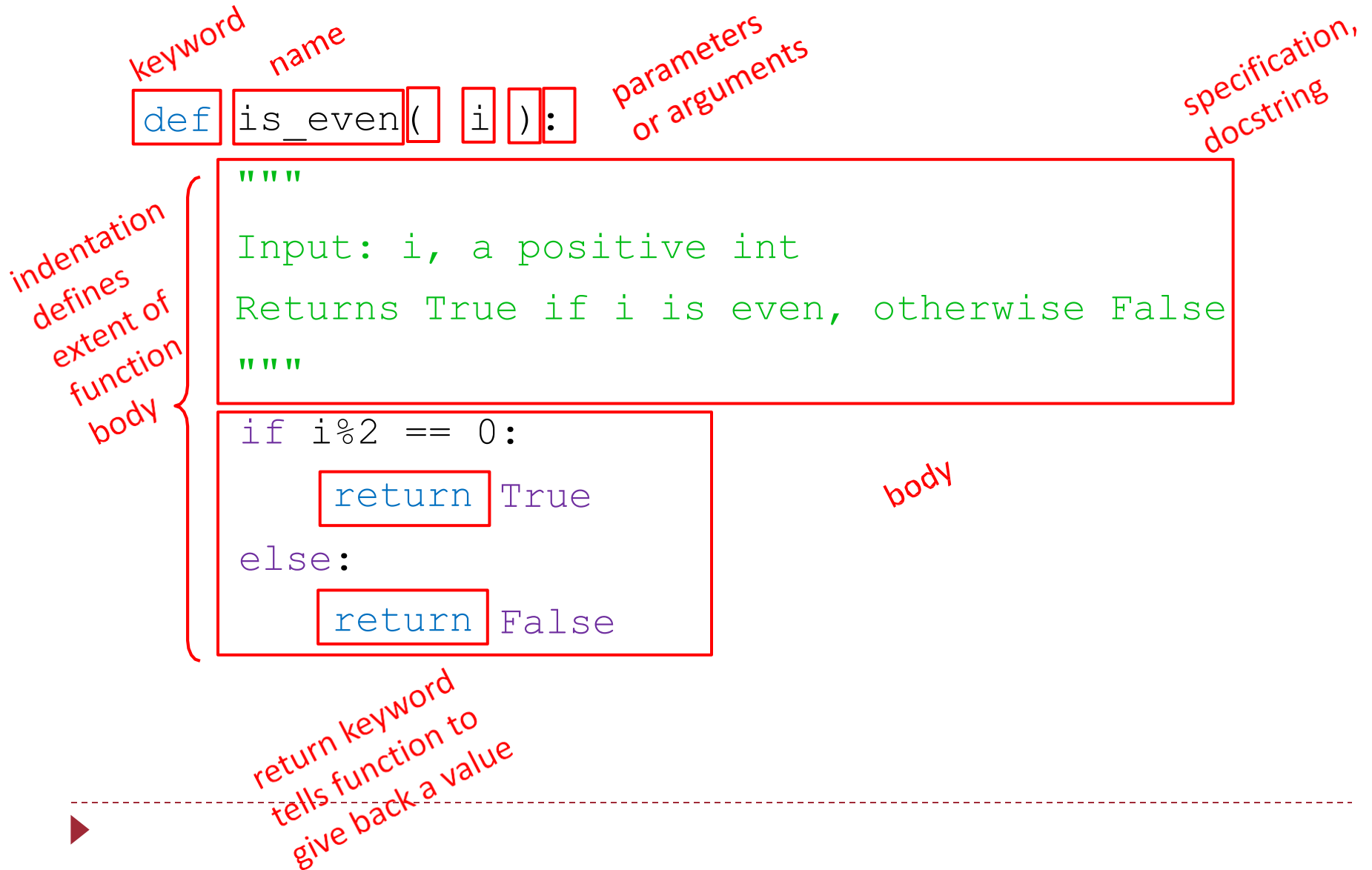


Lecture 5



FUNCTIONs

HOW to WRITE a FUNCTION



HOW TO THINK ABOUT WRITING A FUNCTION

- ▶ **What** is the problem?
 - ▶ Given an int, want to know if it is even
 - ▶ Use this to write the function name and specs

```
def is_even( i ):
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
```



HOW TO THINK ABOUT WRITING A FUNCTION

- ▶ How to **solve** the problem?
 - ▶ Can check that remainder when divided by 2 is 0
 - ▶ Think about what value you need to give back

```
def is_even( i ):
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    if i%2 == 0:
        return True
    else:
        return False
```



HOW TO THINK ABOUT WRITING A FUNCTION

- ▶ Can you make the code cleaner?
 - ▶ $i\%2$ is a Boolean that evaluates to True/False already

```
def is_even( i ):
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    return i%2 == 0
```



HOW TO CALL (INVOKE) A FUNCTION

Name of the function

```
is_even(3)
```

```
is_even(8)
```

▶ That's all!



HOW TO CALL (INVOKE) A FUNCTION

Name of the function

*Values for parameters
of the function*

`is_even(3)`

`is_even(8)`

► That's all!



HOW TO CALL (INVOKE) A FUNCTION

```
is_even(3)
```

```
is_even(8)
```

Replaced by the return!

► That's all!



ALL TOGETHER IN A FILE

- ▶ This code might be in one file

```
def is_even( i ):  
    return i%2 == 0
```

Function definition

```
is_even(3)
```

Function call



WHAT HAPPENS when you CALL a FUNCTION?

► Python binds:

formal parameters in function def with **values from function call**
i bound with **3**

```
def is_even( i ):
    return i%2 == 0
```

i mapped to 3

```
is_even(3)
```



WHAT HAPPENS when you CALL a FUNCTION?

- ▶ Python **executes expressions in the body** of the function

- ▶ `return 3%2 == 0`

```
def is_even( i ):  
    return i%2 == 0
```

expression to evaluate
and return to invoker
`3%2 == 0` is False

```
is_even(3)
```



WHAT HAPPENS when you CALL a FUNCTION?

- ▶ Python replaces the **function call** with the **return value**!

```
def is_even( i ):  
    return i%2 == 0
```

expression to evaluate
and return to invoker
 $3\%2 == 0$ is False

```
is_even(3)
```

Replaced by False



WHAT HAPPENS when you CALL a FUNCTION?

- ▶ Python replaces the **function call** with the **return value**!

```
def is_even( i ):  
    return i%2 == 0
```

```
is_even(3)  
print(is_even(3))
```

Replaced by False



BIG IDEA

A function's code only runs
when you call (aka invoke) the
function



YOU TRY IT!

- ▶ Write code that satisfies the following specs

```
def div_by(n, d):  
    """ n and d are ints > 0  
    Returns True if d divides n evenly and False otherwise  
    """
```

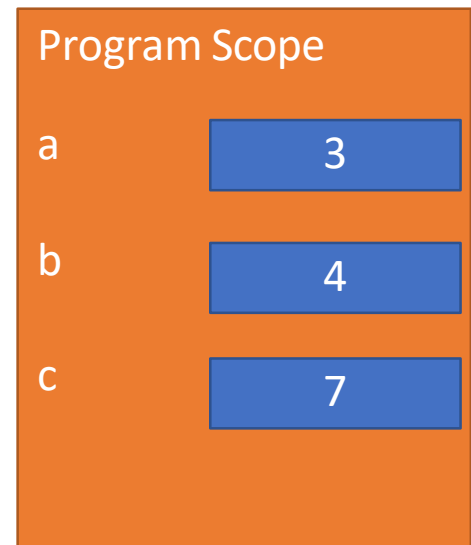
- ▶ Test your code with:
 - ▶ n = 10 and d = 3
 - ▶ n = 195 and d = 13



ZOOMING OUT

(no functions)

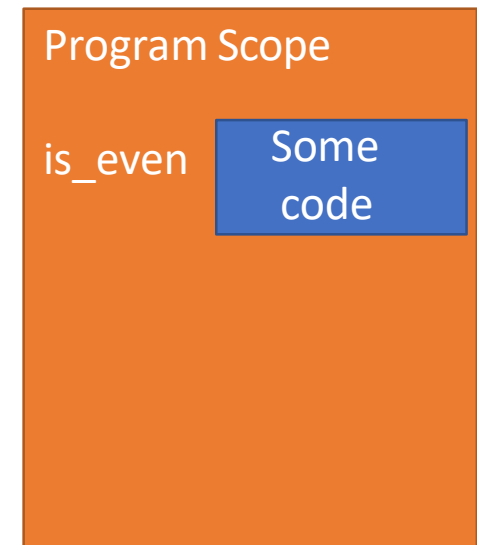
```
a = 3  
b = 4  
c = a+b
```



ZOOMING OUT

```
def is_even( i ):
    print("inside is_even")
    return i%2 == 0
```

```
a = is_even(3)
b = is_even(10)
c = is_even(123456)
```

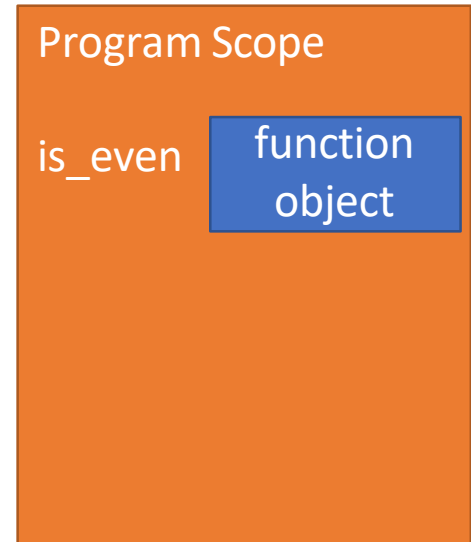


ZOOMING OUT

This is my “black box”

```
def is_even( i ):
    print("inside is_even")
    return i%2 == 0
```

```
a = is_even(3)
b = is_even(10)
c = is_even(123456)
```



ZOOMING OUT

This is my “black box”

```
def is_even( i ):
    print("inside is_even")
    return i%2 == 0
```

```
a = is_even(3)
b = is_even(10)
c = is_even(123456)
```

Program Scope

is_even

function
object

This is me telling my black box to do something

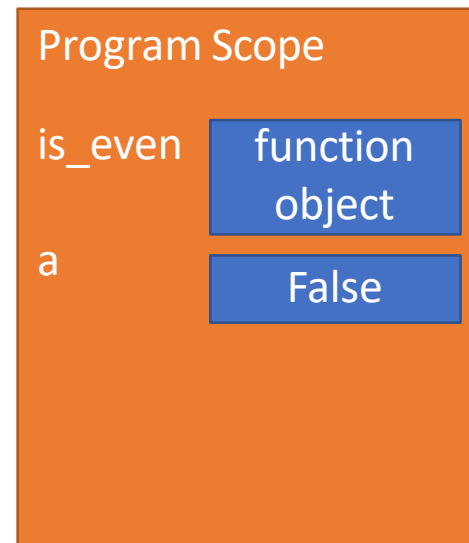
ZOOMING OUT

This is my “black box”

```
def is_even( i ):
    print("inside is_even")
    return i%2 == 0
```

```
a = is_even(3)
b = is_even(10)
c = is_even(123456)
```

One function call



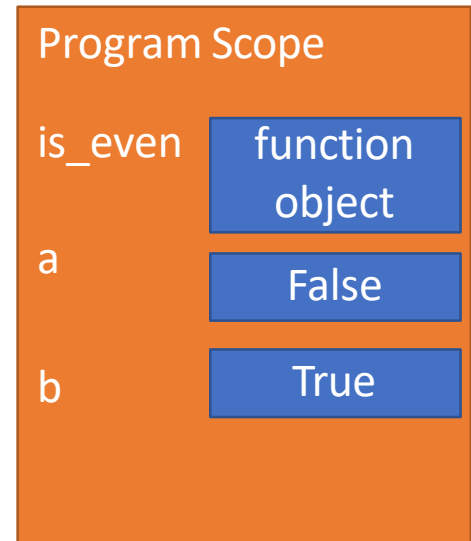
ZOOMING OUT

This is my “black box”

```
def is_even( i ):
    print("inside is_even")
    return i%2 == 0
```

```
a = is_even(3)
b = is_even(10)
c = is_even(123456)
```

One function call



ZOOMING OUT

This is my “black box”

```
def is_even( i ):
    print("inside is_even")
    return i%2 == 0
```

```
a = is_even(3)
```

```
b = is_even(10)
```

```
c = is_even(123456)
```

One function call

Program Scope

is_even	function object
a	False
b	True
c	True



EXAMPLE

- ▶ Suppose we want to add all the odd integers between (and including) a and b
- ▶ What is the **input**?
 - ▶ Values for a and b
- ▶ What is the **output**?
 - ▶ The sum_of_odds

```
def sum_odd(a, b):  
  
    return sum_of_odds
```



BIG IDEA

Don't write code right away!



PAPER FIRST

- ▶ Suppose we want to add all the odd integers between (and including) a and b
- ▶ Start with a **simple example on paper**
- ▶ Systematically solve the example

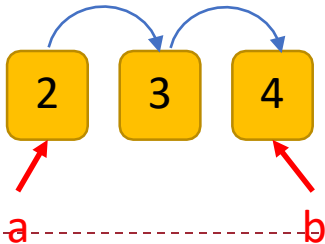
```
def sum_odd(a, b):  
    # your code here  
    return sum_of_odds
```



SIMPLE TEST CASE

- ▶ Suppose we want to add all the odd integers between (and including) a and b
- ▶ Start with a simple example on paper
- ▶ $a = 2$ and $b = 4$
 - ▶ `sum_of_odds` should be 3

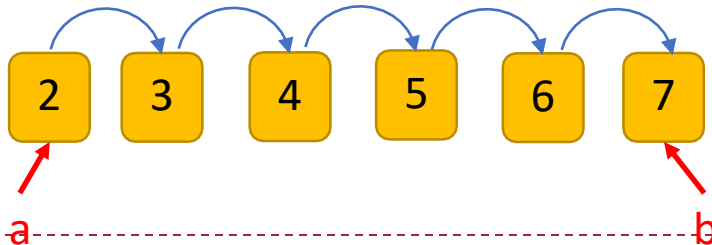
```
def sum_odd(a, b):  
    # your code here  
    return sum_of_odds
```



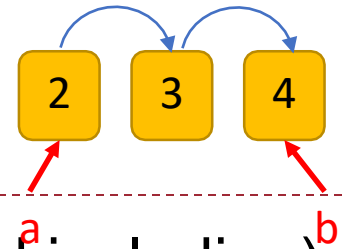
MORE COMPLEX TEST CASE

- ▶ Suppose we want to add all the odd integers between (and including) a and b
- ▶ Start with a simple example on paper
- ▶ $a = 2$ and $b = 7$
 - ▶ `sum_of_odds` should be 15

```
def sum_odd(a, b):  
    # your code here  
    return sum_of_odds
```



SOLVE SIMILAR PROBLEM

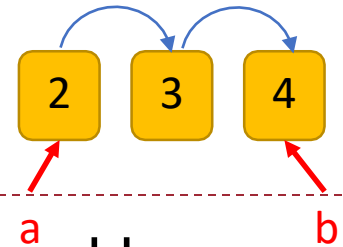


- ▶ Start by looking at each number between (and including) a and b
- ▶ A similar problem that is easier that you know how to do?
 - ▶ Add **ALL** numbers between (and including) a and b
 - ▶ Start with this

```
def sum_odd(a, b):  
    # your code here  
    return sum_of_odds
```



CHOOSE BIG-PICTURE STRUCTURE



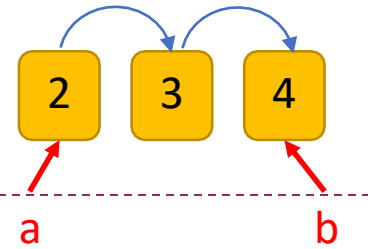
- ▶ Add ALL numbers between (and including) a and b
 - ▶ It's a loop
- ▶ while or for?
 - ▶ Your choice

```
def sum_odd(a, b):  
    # your code here  
    return sum_of_odds
```



WRITE the LOOP

(for adding all numbers)



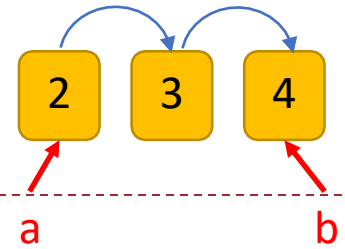
for LOOP

```
def sum_odd(a, b):  
    for i in range(a, b):  
        # do something  
    return sum_of_odds
```

while LOOP

```
def sum_odd(a, b):  
    i = a  
    while i <= b:  
        # do something  
        i += 1  
    return sum_of_odds
```

DO the SUMMING (for adding all numbers)



for LOOP

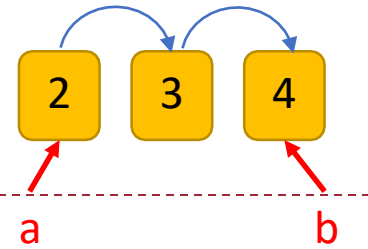
```
def sum_odd(a, b):  
    for i in range(a, b):  
        sum_of_odds += i  
    return sum_of_odds
```

while LOOP

```
def sum_odd(a, b):  
    i = a  
    while i <= b:  
        sum_of_odds += i  
        i += 1  
    return sum_of_odds
```



INITIALIZE the SUM (for adding all numbers)



for LOOP

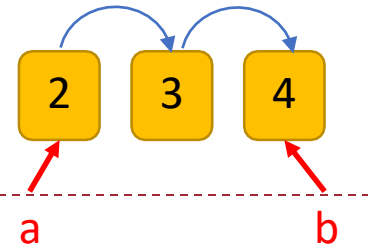
```
def sum_odd(a, b):  
    sum_of_odds = 0  
    for i in range(a, b):  
        sum_of_odds += i  
    return sum_of_odds
```

while LOOP

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    i = a  
    while i <= b:  
        sum_of_odds += i  
        i += 1  
    return sum_of_odds
```

TEST!

(for adding all numbers)



for **LOOP**

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    for i in range(a, b):  
        sum_of_odds += i  
    return sum_of_odds
```

```
print(sum_odd(2, 4))
```

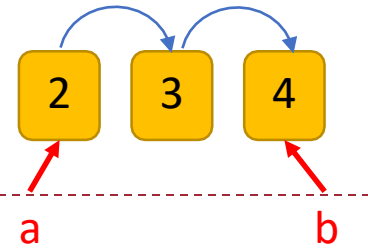
while **LOOP**

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    i = a  
    while i <= b:  
        sum_of_odds += i  
        i += 1  
    return sum_of_odds
```

```
print(sum_odd(2, 4))
```

WEIRD RESULTS...

(for adding all numbers)



for LOOP

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    for i in range(a, b):  
        sum_of_odds += i  
    return sum_of_odds
```

```
print(sum_odd(2, 4))
```

5

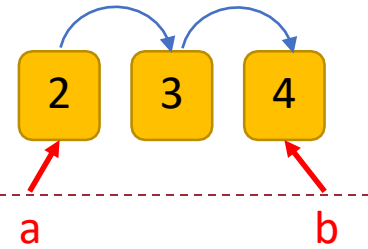
while LOOP

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    i = a  
    while i <= b:  
        sum_of_odds += i  
        i += 1  
    return sum_of_odds
```

```
print(sum_odd(2, 4))
```

9

DEBUG! aka ADD PRINT STATEMENTS (for adding all numbers)



for LOOP

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    for i in range(a, b):  
        sum_of_odds += i  
        print(i, sum_of_odds)  
    return sum_of_odds
```

```
print(sum_odd(2, 4))
```

```
2 2  
3 5
```

5

while LOOP

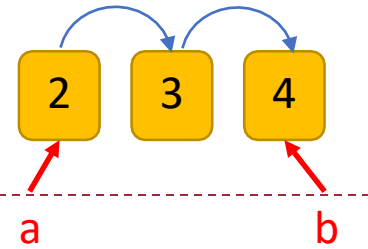
```
def sum_odd(a, b):  
    sum_of_odds = 0  
    i = a  
    while i <= b:  
        sum_of_odds += i  
        print(i, sum_of_odds)  
        i += 1  
    return sum_of_odds
```

```
print(sum_odd(2, 4))
```

```
2 2  
3 5  
4 9
```

9

FIX for LOOP END INDEX (for adding all numbers)



for LOOP

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    for i in range(a, b+1):  
        sum_of_odds += i  
        print(i, sum_of_odds)  
    return sum_of_odds
```

```
print(sum_odd(2, 4))
```

9

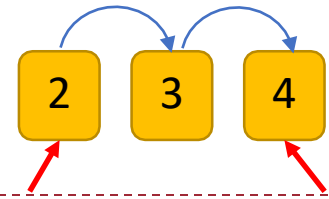
while LOOP

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    i = a  
    while i <= b:  
        sum_of_odds += i  
        print(i, sum_of_odds)  
        i += 1  
    return sum_of_odds
```

```
print(sum_odd(2, 4))
```

9

ADD IN THE ODD PART!



for LOOP

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    for i in range(a, b+1):  
        if i%2 == 1:  
            sum_of_odds += i  
            print(i, sum_of_odds)  
    return sum_of_odds
```

```
print(sum_odd(2,4))
```

3

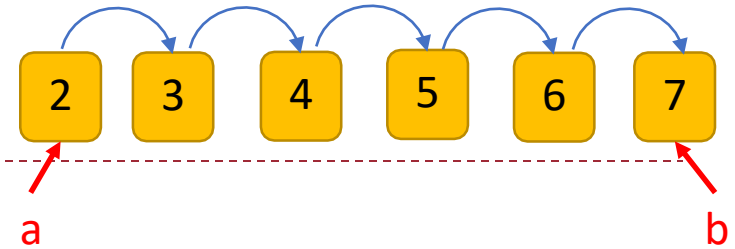
while LOOP

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    i = a  
    while i <= b:  
        if i%2 == 1:  
            sum_of_odds += i  
            print(i, sum_of_odds)  
        i += 1  
    return sum_of_odds
```

```
print(sum_odd(2,4))
```

3

TRY IT ON ANOTHER EXAMPLE



for LOOP

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    for i in range(a, b+1):  
        if i%2 == 1:  
            sum_of_odds += i  
    return sum_of_odds
```

```
print(sum_odd(2, 7))
```

15

while LOOP

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    i = a  
    while i <= b:  
        if i%2 == 1:  
            sum_of_odds += i  
        i += 1  
    return sum_of_odds
```

```
print(sum_odd(2, 7))
```

15

BIG IDEA

Solve a simpler problem first.
Add functionality to the code later.



BIG IDEA

Test code often.

Use prints to debug.



return KEYWORD

```
def is_even( i ):  
    """  
    Input: i, a positive int  
    Returns True if i is even and False otherwise  
    """  
    return i%2 == 0
```

- ▶ A function always returns something



WHAT IF THERE IS NO `return` KEYWORD

```
def is_even( i ):  
    """  
    Input: i, a positive int  
    Does not return anything  
    """
```

`i % 2 == 0`

*without a return
statement*

- ▶ Python returns the value **None, if no return given**
- ▶ Represents the absence of a value
 - ▶ If invoked in shell, nothing is printed
- ▶ No static semantic error generated



WHAT IF THERE IS NO `return` KEYWORD

```
def is_even( i ):
```

```
    """
```

```
    Input: i, a positive int
```

```
    Does not return anything
```

```
    """
```

```
    i%2 == 0
```

return	None
--------	------

A line Python adds
implicitly
(do not add it yourself)

None is a value of type `NoneType`
(not a string, not a number, etc)



WHAT IF THERE IS NO `return` KEYWORD

```
def is_even( i ):  
    """  
    Input: i, a positive int  
    Does not return anything  
    """  
  
    i%2 == 0  
    return
```



YOU TRY IT!

- ▶ What is printed if you run this code as a file?

```
def add(x, y):  
    return x+y  
def mult(x, y):  
    print(x*y)
```

```
add(1, 2)
```

```
print(add(2, 3))
```

```
mult(3, 4)
```

```
print(mult(4, 5))
```



FUNCTIONS SUPPORT MODULARITY

- Here is our bisection square root method as a function

```
def bisection_root(x):
```

```
    epsilon = 0.01  
    low = 0  
    high = x  
    ans = (high + low)/2.0
```

Initialize variables

```
    while abs(ans**2 - x) >= epsilon:
```

guess not close enough

```
        if ans**2 < x:
```

```
            low = ans
```

update low or high,
depends on guess too
small or too large

```
        else:
```

```
            high = ans
```

```
        ans = (high + low)/2.0
```

new value for guess

```
    # print(ans, 'is close to the root of', x)
```

```
    return ans
```

return result

iterate



FUNCTIONS SUPPORT MODULARITY

- ▶ Call it with different values

```
print(bisection_root(4))  
print(bisection_root(123))
```

- ▶ Write a function that calls this one!



YOU TRY IT!

- ▶ Write a function that satisfies the following specs

```
def count_nums_with_sqrt_close_to (n,epsilon):  
    """ n is an int > 2  
        epsilon is a positive number < 1  
        Returns how many integers have a square root within epsilon of n  
    """
```

- ▶ Use `bisection_root` we already wrote to get an approximation for the sqrt of an integer.
- ▶ For example: `print(count_nums_with_sqrt_close_to(10, 0.1))` prints 4 because all these integers have a sqrt within 0.1
 - ▶ sqrt of 99 is 9.949699401855469
 - ▶ sqrt of 100 is 9.999847412109375
 - ▶ sqrt of 101 is 10.049758911132812
 - ▶ sqrt of 102 is 10.099456787109375



FUNCTIONS: DEFAULT PARAMETERS

SQUARE ROOT with BISECTION

```
def bisection_root(x):  
    epsilon = 0.01  
    low = 0  
    high = x  
    guess = (high + low)/2.0  
    while abs(guess**2 - x) >= epsilon:  
        if guess**2 < x:  
            low = guess  
        else:  
            high = guess  
        guess = (high + low)/2.0  
    return guess  
  
print(bisection_root(123))
```



epsilon as a PARAMETER

```
def bisection_root(x, epsilon):  
    low = 0  
    high = x  
    guess = (high + low)/2.0  
    while abs(guess**2 - x) >= epsilon:  
        if guess**2 < x:  
            low = guess  
        else:  
            high = guess  
        guess = (high + low)/2.0  
    return guess  
  
print(bisection_root(123, 0.01))
```



DEFAULT PARAMETER

```
def bisection_root(x, epsilon)
```

- ▶ We added epsilon as an argument to the function
 - ▶ Most of the time we want some standard value, 0.01
 - ▶ Sometimes, we may want to use some other value
- ▶ Use a default parameter



Epsilon as a DEFAULT PARAMETER

```
def bisection_root(x, epsilon=0.01):
```

```
    low = 0
```

```
    high = x
```

```
    guess = (high + low)/2.0
```

```
    while abs(guess**2 - x) >= epsilon:
```

```
        if guess**2 < x:
```

```
            low = guess
```

```
        else:
```

```
            high = guess
```

```
        guess = (high + low)/2.0
```

```
    return guess
```

```
print(bisection_root(123))
```

```
print(bisection_root(123, 0.5))
```

*Default parameter, with
default value of 0.01*

*Uses epsilon as 0.01 (the default one in
function def)*

Uses epsilon as 0.5



RULES for FUNCTION PARAMETERS

- ▶ In the **function definition**,
 - ▶ Default parameters must go at the end

- ▶ These are **ok for calling a function**:

- ▶ `bisection_root_new(123)`
- ▶ `bisection_root_new(123, 0.001)`
- ▶ `bisection_root_new(123, epsilon=0.001)`
- ▶ `bisection_root_new(x=123, epsilon=0.1)`
- ▶ `bisection_root_new(epsilon=0.1, x=123)`

Positional arguments



Keyword arguments



- ▶ These are **not ok for calling a function**:

- ▶ `bisection_root_new(epsilon=0.001, 123)`
 - ▶ Error: positional argument shall precede keyword arguments
- ▶ `bisection_root_new(0.001, 123)`
 - ▶ No error but wrong





FUNCTION SCOPE

ZOOMING OUT

This is my “black box”

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    for i in range(a, b+1):  
        if i%2 == 1:  
            sum_of_odds += i  
    return sum_of_odds
```

```
low = 2  
high = 7  
my_sum = sum_odd(low, high)
```

One function call

Program Scope

sum_odd	function object
low	2
high	7
my_sum	



ZOOMING OUT

a=2 b=7

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    for i in range(a, b+1):  
        if i%2 == 1:  
            sum_of_odds += i  
    return sum_of_odds
```

```
low = 2  
high = 7  
my_sum = sum_odd(low, high)
```

Program Scope

sum_odd	function object
low	2
high	7
my_sum	



ZOOMING OUT

This is my “black box”

```
def sum_odd(a, b):  
    sum_of_odds = 0  
    for i in range(a, b+1):  
        if i%2 == 1:  
            sum_of_odds += i  
    return sum_of_odds
```

```
low = 2  
high = 7  
my_sum = sum_odd(low, high)
```

Program Scope

sum_odd	function object
low	2
high	7
my_sum	15



UNDERSTANDING FUNCTION CALLS

- ▶ How does Python execute a function call?
- ▶ How does Python know what value is associated with a variable name?
- ▶ It **creates a new environment** with every function call!
 - ▶ Within the environment, it **assigns its parameters** to some inputs
 - ▶ It does the work (aka the **body** of the function)
 - ▶ It **returns** a value
- ▶ The **environment disappears** after it returns the value



ENVIRONMENTS

- ▶ **Environment / Scope** contains a mapping of names to objects
 - ▶ Values of variables given by bindings of names
- ▶ **Global** environment
 - ▶ Where user interacts with Python interpreter
 - ▶ Where the program starts out
- ▶ Invoking a function creates a **new environment** (frame/scope)



VARIABLE SCOPE

after evaluating def

This is my “black box”

```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

Global scope

f

function
object

➡
x = 3

z = f(x)

VARIABLE SCOPE

after exec 1st assignment

This is my “black box”

```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

Global scope

f

Some
code

x

3

x = 3

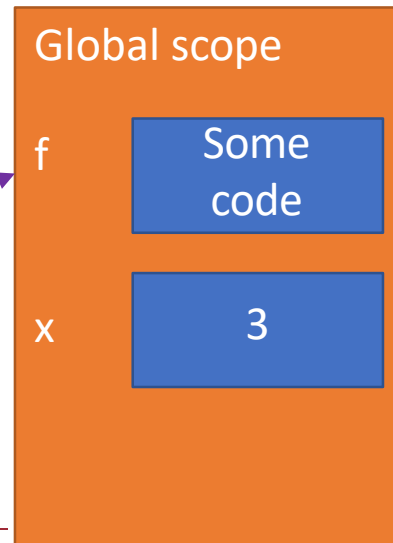
z = f(x)

VARIABLE SCOPE

after f invoked

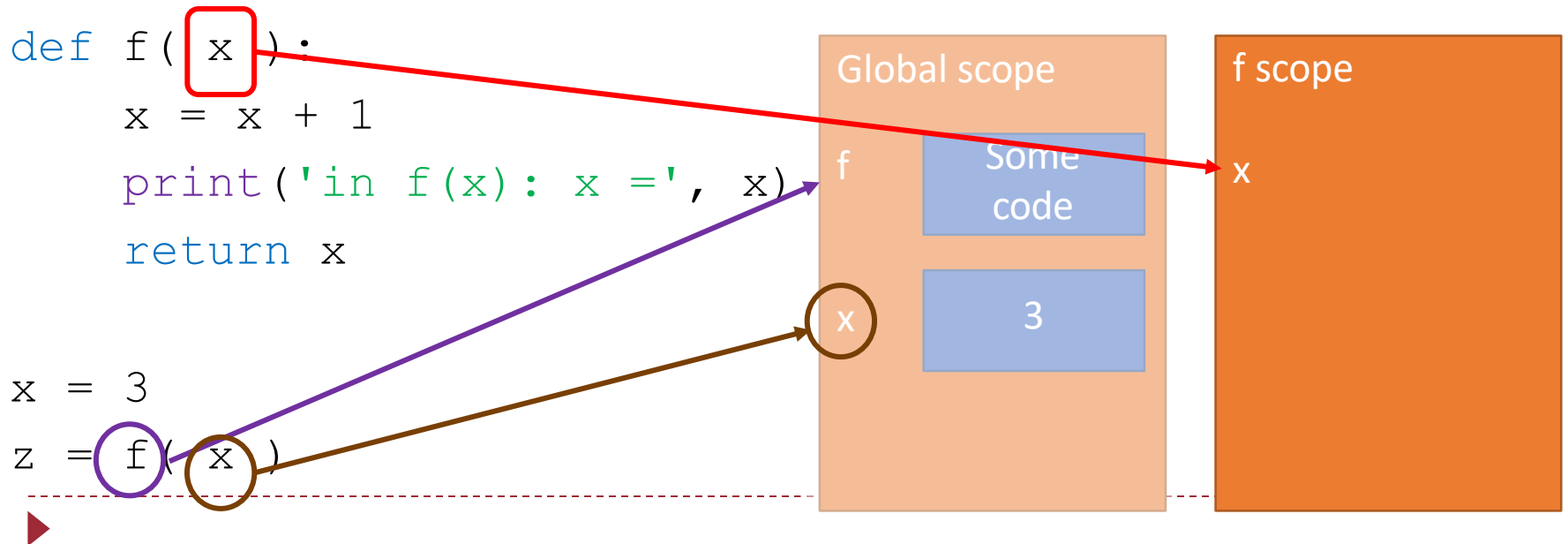
```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

```
x = 3  
z = f( x )
```



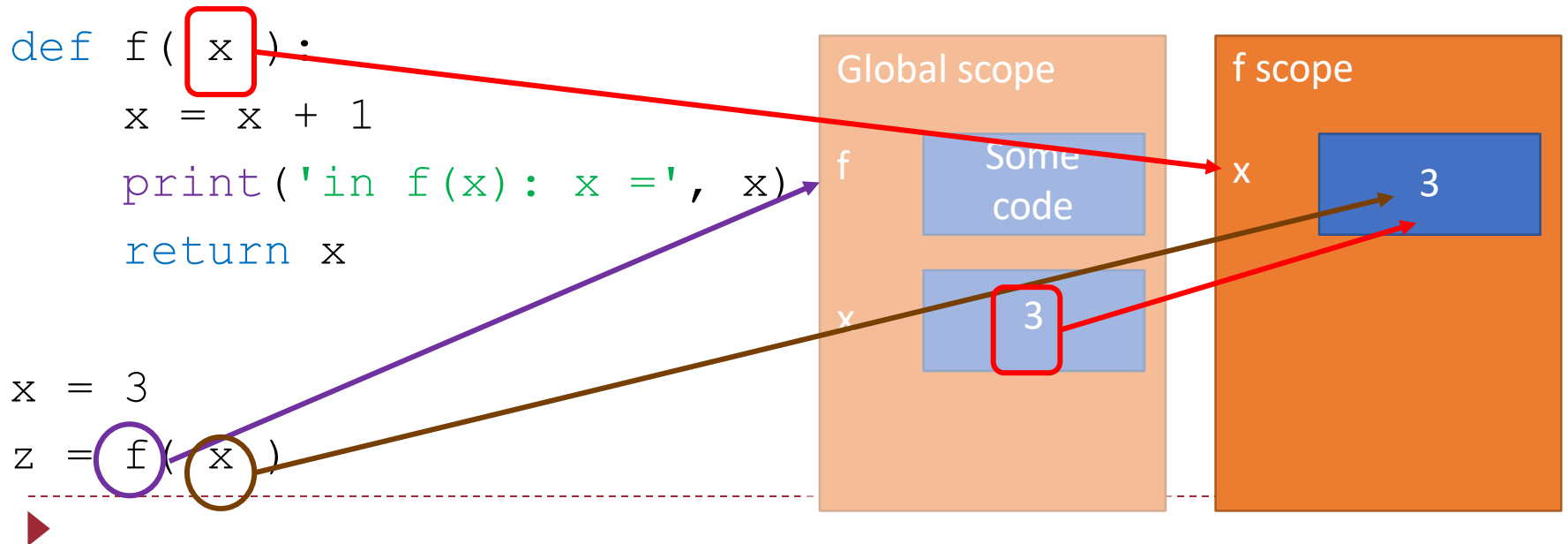
VARIABLE SCOPE

after f invoked



VARIABLE SCOPE

after f invoked



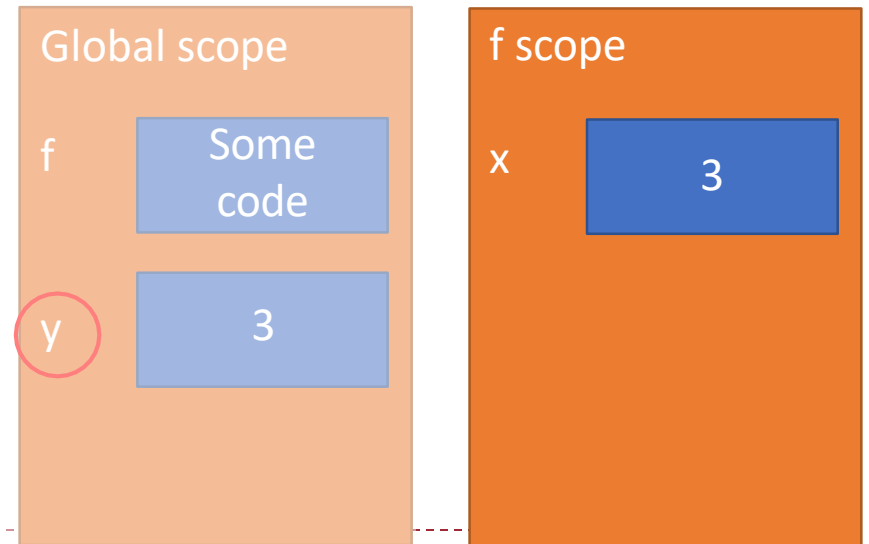
VARIABLE SCOPE

after f invoked

Name of variable irrelevant, only value important. You can also pass in the value directly.

```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

y = 3
→ z = f(**y**)

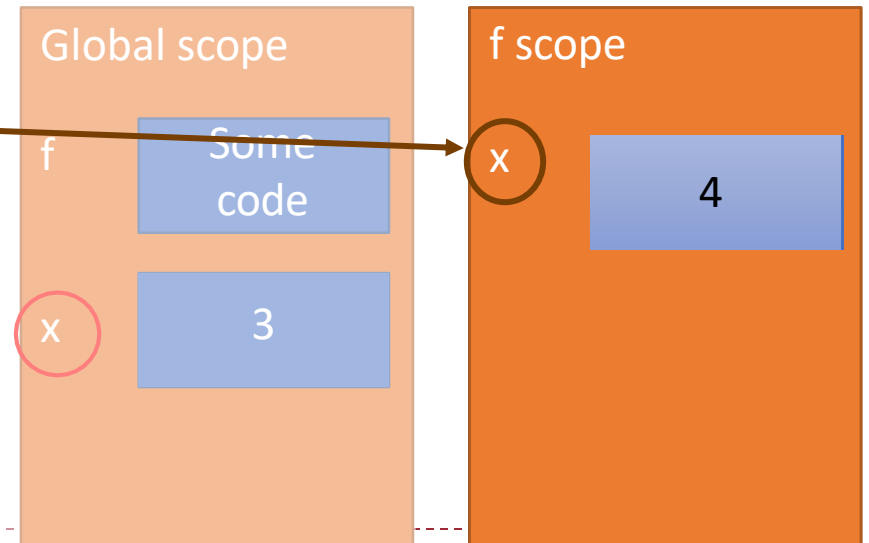


VARIABLE SCOPE

eval body of f in f's scope

```
def f( x ):  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

```
x = 3  
z = f( x )
```



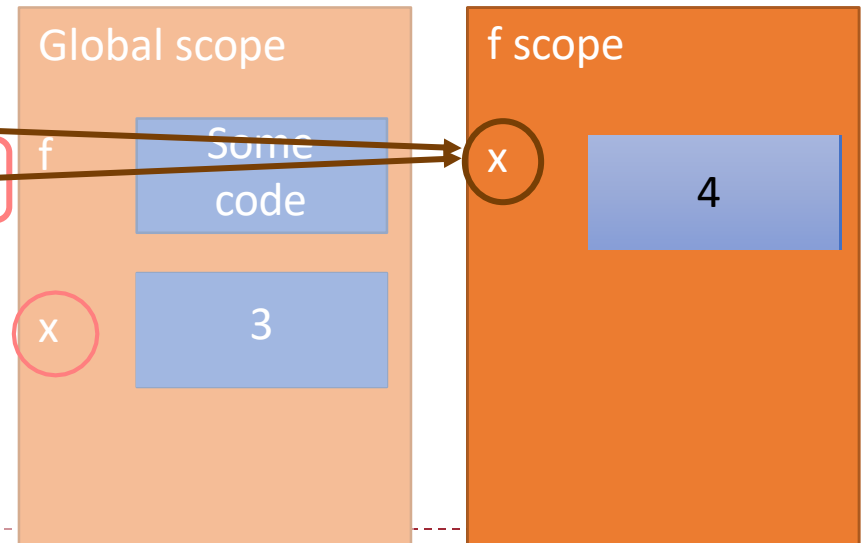
VARIABLE SCOPE

eval body of f in f's scope

in f(x): x = 4 printed out

```
def f( x ):  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

```
x = 3  
z = f( x )
```



VARIABLE SCOPE

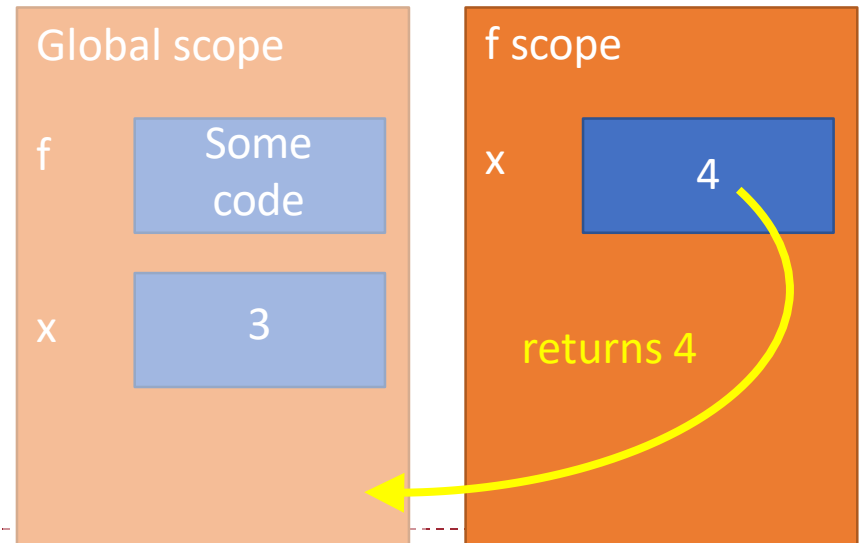
during return

```
def f( x ):  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

```
x = 3
```

```
z = f( x )
```

Function call
replaced with
return value



VARIABLE SCOPE

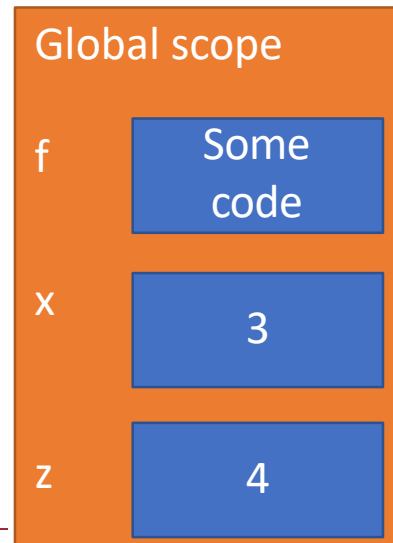
after exec 2nd assignment

*If I now ask for value of x in
Python interpreter, it will print 3*

```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

```
x = 3
```

```
z = f( x )
```



VARIABLE SCOPE

- ▶ Inside a function, **can access** a variable defined outside
- ▶ Inside a function, **cannot modify** a variable defined outside (can by using **global variables**, but frowned upon)

```
def f(y):  
    x = 1  
    x += 1  
    print(x)
```

```
x = 5  
f(x)  
print(x)
```

```
def g(y):  
    print(x)  
    print(x + 1)
```

```
x = 5  
g(x)  
print(x)
```

```
def h(y):  
    x += 1
```

```
x = 5  
h(x)  
print(x)
```



VARIABLE SCOPE

- ▶ Inside a function, **can access** a variable defined outside
- ▶ Inside a function, **cannot modify** a variable defined outside (can by using **global variables**, but frowned upon)

```
def f(y):  
    x = 1  
    x += 1  
    print(x)  
  
x = 5  
f(x)  
print(x)
```

x is re-defined in scope of f

different x objects

2
5

```
def g(y):  
    print(x)  
    print(x + 1)  
  
x = 5  
g(x)  
print(x)
```

```
def h(y):  
    x += 1  
  
x = 5  
h(x)  
print(x)
```

VARIABLE SCOPE

- ▶ Inside a function, **can access** a variable defined outside
- ▶ Inside a function, **cannot modify** a variable defined outside (can by using **global variables**, but frowned upon)

```
def f(y):  
    x = 1  
    x += 1  
    print(x)
```

x is re-defined in scope of f

```
    x = 5  
    f(x)  
    print(x)
```

```
2  
5
```

different x objects

```
def g(y):  
    print(x)  
    print(x + 1)
```

x from outside g

```
    x = 5  
    g(x)  
    print(x)
```

```
5  
6  
5
```

x inside g is picked up from scope that called function g

```
def h(y):  
    x += 1
```

```
    x = 5  
    h(x)  
    print(x)
```



VARIABLE SCOPE

- ▶ Inside a function, **can access** a variable defined outside
- ▶ Inside a function, **cannot modify** a variable defined outside (can by using **global variables**, but frowned upon)

```
def f(y):  
    x = 1  
    x += 1  
    print(x)  
  
x = 5  
f(x)  
print(x)
```

x is re-defined in scope of f

different x objects

2
5

```
def g(y):  
    print(x)  
    print(x + 1)  
  
x = 5  
g(x)  
print(x)
```

x from outside g

x inside g is picked up from scope that called function g

5
6
5

```
def h(y):  
    x += 1  
  
x = 5  
h(x)  
print(x)
```

UnboundLocalError: local variable 'x' referenced before assignment

Error

VARIABLE SCOPE

▶ LEGB Rule

- ▶ **Local** — Names assigned in any way within a function, and not declared global in that function
- ▶ **Enclosing-function** — Names assigned in the local scope of any and all statically enclosing functions, from inner to outer
- ▶ **Global** — Names assigned at the top-level of a module file, or by executing a global statement in a def within the file
- ▶ **Built-in** — Names preassigned in the built-in names module: len, abs, range, etc





FUNCTIONS as ARGUMENTS

HIGHER ORDER PROCEDURES

- ▶ Objects in Python have a type
 - ▶ int, float, str, Boolean, NoneType, function
- ▶ Objects can appear in RHS of assignment statement
 - ▶ Bind a name to an object
- ▶ Objects
 - ▶ Can be **used as an argument** to a procedure
 - ▶ Can be **returned as a value** from a procedure
- ▶ Functions are also **first class objects**!
- ▶ Treat functions just like the other types
 - ▶ Functions can be bound to new names
 - ▶ Functions can be arguments to another function
 - ▶ Functions can be returned by another function



OBJECTS IN A PROGRAM

```
def is_even(i):  
    return i%2 == 0
```

```
r = 2
```

```
pi = 22/7
```

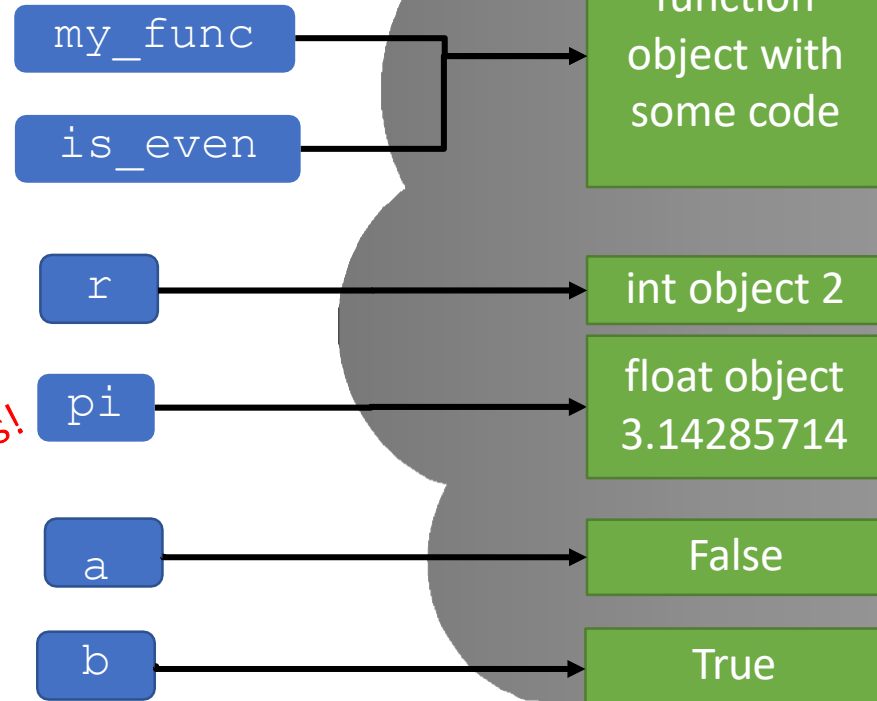
```
my_func = is_even
```

```
a = is_even(3)
```

```
b = my_func(4)
```

*NOT a function
call, just names!*

Two function calls



BIG IDEA

- ▶ Everything in Python is an object.

